CSC 3320: System-Level Programming

Georgia State University

Chapter 16: Structures, Unions, and Enumerations

Instructor: Dr. Md Mahfuzur Rahman

Department of Computer Science, GSU

Note: These slides are updated/improved by M. Rahman on top of original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Structure

- C has no class concept, but C has structure.
- A **structure** is a collection of values(members), possibly of different types.
- In other languages, structures are often called **records**; members of structures are known as **fields**.
- The members of a structure are stored in memory in the order in which they are declared.

Declaring Structure Variable

```
struct {
    int number;
    char name[NAME_LEN + 1];
    int on_hand;
} part1, part2;
```

- `struct{...}` specifies a type
- `part1` and `part2` are variables of that type
- `number` , `name` array and `on_hand` are members of this structure

Structure Initialization

```
struct {
    int number;
    char name[NAME_LEN+1];
    int on_hand;
} partl = {528, "Disk drive", 10}, part2 =
{914, "Printer cable", 5);
```

Operations on Structures

- Use period '.' to access a member within a structure

```
printf("Part number : %d \n", part1.number);


part1.number = 258;
part1.on_hand++;

scanf("%d",&part1.on_hand);   // &(part1.on_hand)
```

- Use equal sign '=' to copy structures

```
part1 = part2 // The members in part 1 and part 2
              //then have the same values
```

Structure Types

- Declaring a structure tag

```
struct part {
    int number;
    char name[NAME_LEN  + 1];
    int on_hand;
}

struct part part1, part2;

part part1, part2; // WRONG
```

- Declaring a structure type

```
typedef struct {
    int number;
    char name[NAME_LEN + 1];
    int on_hand;
} Part;

 Part part1, part2;
```

Structures as Arguments

- Pass by values

```
 void print_part (struct part p)
{
        printf("Part number: %d\n", p.number);
}
```

- Pass by addresses

```
 void update_part (struct part *p)
{
        p->number = 123;     // (*p).number = 210; is valid
}
```

For pointers to a structure, use '->' to access to the members instead of period '.'

Example code

```c
#include<stdio.h>
struct Part
{
    int number;
};

void update(struct Part *p)
{
    (*p).number = 210;  // p->number = 210 should also work
}

void display(struct Part p)
{
    printf("\nNumber = %d", p.number);
}

int main()
{
    struct Part p1={.number=120}, p2 ={.number=205}; // initialization or p1={120}

    display(p1);
    display(p2);
    update(&p1);
    display(p1);

    return 0;
}
Output: Number = 120\nNumber = 205\nNumber = 210
```

Structures as Return Values


```
struct part build_part (int number,
                        const char * name, int on_hand)
 {
        struct part p;
        p.number = number;
        strcpy(p.name, name);
        p.on_hand= on_hand;
        return p;
 }
```

Arrays of Structures

```
struct part  inventory[100]; // array of part

print_part(inventory[i]); // print the ith part

inventory[i].number = 883;
// assign 883 to the number member of inventory[i]

inventory[i].name = "Disk Drive";
// assign "Disk Drive" to the name member of
inventory[i]
```

Nested Structures

```
struct person_name {
    char first[FIRST_NAME_LEN+l];
    char middle_initial;
    char last[LAST_NAME_LEN+l];
};
```

We can use the person_name structure as part of a larger structure:

```
struct student {
    struct person_name name;
    int id, age;
    char sex;
} studentl, student2;
```

```
strcpy(studentl.name.first, "Fred");
```

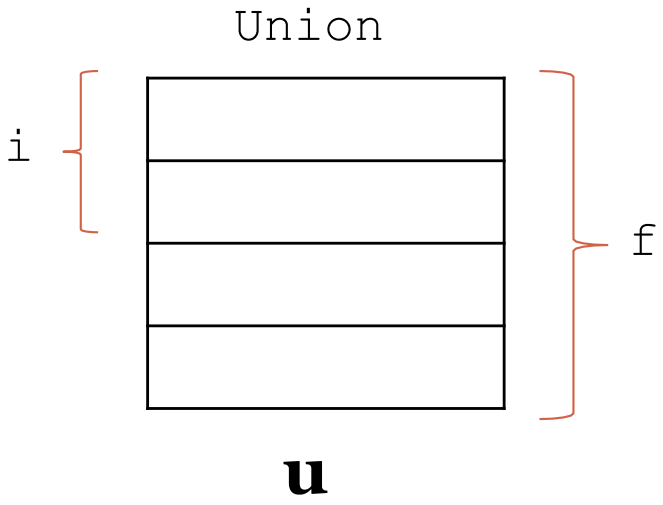
Unions

- A **union**, like a structure, consists of one or more members.
- The difference is that compiler allocates only enough space for the largest of the members in a union.
- The members of the union overlay each other within this space.
- Assigning a new value to one member alters the values of all the other members as well.

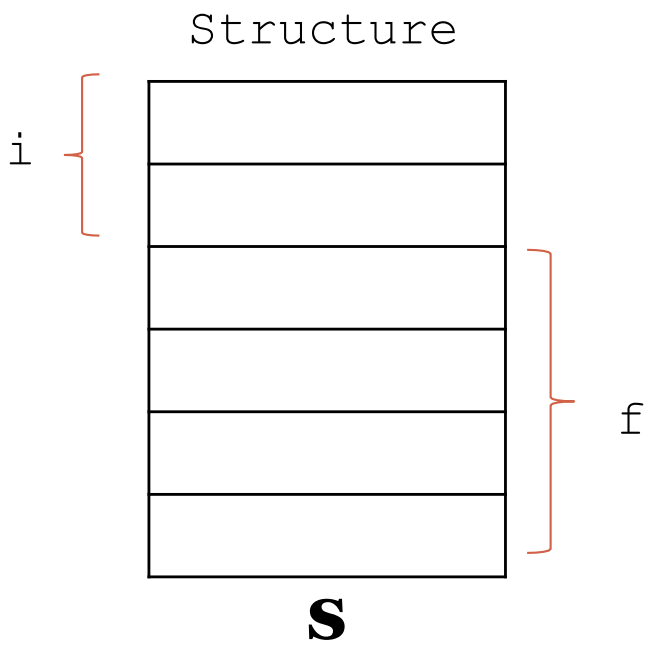
Using Unions to Save Spaces

- Union VS. Structure

```
union{
  int   i;
  float   f;
} u ;
```




```
struct {
  int   i;
  float   f;
} s ;
```



Using Unions to Save Spaces

Georgia State University

- Example

Books: Title , author, number of pages
Mugs: Design
Shirts: Design, colors available , sizes available

```
struct catalog_item {
  int stock_number;
  float price;
  int item_type;
  char title[TITLE_LEN+1];
  char author[AUTHOR_LEN+1];
  int num_pages;
  char design[DESIGN_LEN+1];
  int colors;
  int sizes;
}
```

Using Unions to Save Spaces

Georgia State University

```
struct catalog_item {
  int stock_number;
  float price;
  int item_type;
  union {
     struct {
       char title[TITLE_LEN+1];
       char author[AUTHOR_LEN+1];
       int num_pages;
     } book;
     struct {
       char design[DESIGN_LEN+1];
     } mug;
     struct {
       char design[DESIGN_LEN+1];
       int colors;
       int sizes;
     } shirt;
  } item;
};
```

Example

```c
#include <stdio.h>

int main()
{
    union {
        int i1;
        int i2;
    } myVar = {.i2 = 100};

    printf("%d %d", myVar.i1, myVar.i2);

    return 0;
}
```

What will be printed?

Enumerations

Assume a variable that stores the suit of a playing card having only four potential values: “clubs,” “diamonds,” “hearts,” and “spades.”

One Solution:

```
int s; /* s will store a suit */ •
s = 2; /* 2 represents "hearts” */
```

Another Solution:

```
#define SUIT int
#define CLUBS 0
#define DIAMONDS 1
#define HEARTS 2
#define SPADES 3
```

Our previous example now becomes easier to read:

```
SUIT S;
S = HEARTS;
```

Enumerations

- An enumeration is a type whose values are listed ("enumerated") by the programmer.
- Each value has name defined by the programmer.

```
enum  {CLUBS, DIAMONDS, HEARTS, SPADES} s;

int i;
i = DIAMONDS;   /*  i  is now  1            */
s = 0;          /*  s is now 0 (CLUBS)     */
s++;            /*  s is now 1 (DIAMONDS)  */
i = s + 2;      /*  i is now 3             */
```

Excercise

What will be the value of BLACK and DK_GRAY constants?

```
enum colors {BLACK, LT_GRAY = 7, DK_GRAY, WHITE = 15};
```

Exercise

The following structures are designed to store information about objects on a graphics screen:

```
struct point {
        int x;
        int y;
};


struct rectangle {
        struct point upper_left, lower_right;
};
```

Write functions that computes the area of a rectangle structure **r** passed as an argument.

Example

```c
#include<stdio.h>
struct st
{
    int x;
    static int y;
};
int main()
{
    printf("%ld", sizeof(struct st));
    return 0;
}
```

Compiler error: static member is not allowed in C structure